System Design and Software Engineering

Course Outline

1.0 COURSE DETAILS			
Course Title	***System Design and Software Engineering***		
Major	**B.Tech. CS & AI, B.Tech. CS & DS**		
Credits	**4**	**Domain with Course Code**	**CSA234**
Mode	**Regular**	**Period**	**Even Semester 2026**
Academic Year and Semester	**2025-2026 Even Semester**	**Category (Core/Audit)**	**Core**
Instructor	**Deepak**	**Office Hours**	
LTP (Lecture-Tutorial-Practical)	**2-1-2**		

2.0 SUBJECT OVERVIEW *The course description should convey to students the intellectual goals of the course -- e.g., the rationale for the course, the guiding questions for the course, the general content of the course.*	
Facilitator's Vision:	To help students think like software engineers — building systems that are reliable, testable, and maintainable, not just functional. The goal is to move beyond coding into design thinking and structured development.
Relevance for Students:	Students will learn how real-world software is designed, built, and evolved. The course connects concepts from OOP, design patterns, UML, and databases to practical system-building — preparing them for machine coding, internships, and product development.
Uniqueness of the program	*Unlike generic programming courses, this focuses on how professional software is architected — from idea to MVP. Students learn through patterns, UML modeling, and system-level reasoning, blending design, code, and collaboration practices.*
Pre-Requisites:	• Object-Oriented Programming Basics • Data Structures & Algorithms • Introductory Database Concepts
Intellectual Goals & Guiding Questions:	1. What makes software engineering different from programming? 2. How do we design systems that are easy to maintain and scale? 3. How can UML and ER diagrams help bridge ideas to code? 4. What principles and patterns make software more resilient and testable? 5. How do teams collaborate effectively to build large systems?

3.0 EDUCATIONAL OBJECTIVES	
Objectives	***What the program will include to meet the requirements of the program goals.*** By the end of this course, students will be able to: 1. **Understand** Core concepts of software engineering — OOP principles, UML, design patterns, and SDLC. 2. **Apply** SOLID principles, modularization, and testing strategies to design real-world software. 3. **Analyze** System design trade-offs in scalability, reliability, and maintainability. 4. **Evaluate** Different architectures and patterns to choose the right approach for a given problem. 5. **Create** A working MVP or machine-coding project with clean design, documentation, and tests.

4.0 LEARNING OUTCOMES		
Outcomes	*Identify the learning outcomes based on Skills, Knowledge, Attitudes and Values that will be worked upon during the course.*	
	Skills	1. Design and document systems using UML, ER, and class diagrams. 2. Apply OOP principles and design patterns in code. 3. Write unit tests and structure modular, maintainable projects. 4. Use version control and collaborate effectively in teams.
	Knowledge	1. Understand how architecture, design patterns, and principles guide software evolution. 2. Explain how system forces (scalability, reliability, observability) affect design.

		3. Describe the lifecycle of software projects and the role of processes like Agile and DevOps.
	Attitudes/Values	1. Develop an engineering mindset — prioritizing design clarity and maintainability. 2. Appreciate team collaboration, code reviews, and continuous learning. 3. Value disciplined software design and responsible use of AI tools in development.

5.0 PROGRAM OUTLINE

Map the learnings of each week of the course

WEEK	CONTENT	ACTIVITY/OUTCLASS	ASSIGNMENTS/ASSESSMENTS
1	**Setting up the Context for SESD** • Activity to establish that planning and architecture are needed to build resilient and useful systems **What is Software Engineering** • Difference between programming and engineering • Software quality attributes — reliability, testability, maintainability • Introduction to software project lifecycle and team roles		
2	**Classes and Objects** • Classes vs Objects — state, behavior, and identity • Constructors and invariants • Basic UML class notation — classes, attributes, methods, and associations **OOPs Introduction** • What is Object-Oriented Programming and why we use it • Inheritance — when and why to use; pitfalls; composition vs inheritance		

	• UML concepts — generalization, interfaces, and realization		
3	**OOPs (Part 1)** • Polymorphism — subtype and interface polymorphism • Abstraction — interfaces and contracts • Intention-revealing APIs and contracts **OOPs (Part 2)** • Encapsulation — protecting invariants, visibility, DTO vs domain object • Scalability, Reliability, and Observability — how these forces shape class and module boundaries (intuition only)		
4	**UML** • Introduction to UML — Behavioral vs Structural diagrams • Use Case Diagrams — actors, boundaries, and goals • Translating requirements into initial UML models **DBMS Recap** • Relational model — keys and basic normalization • Indexing and common query patterns (quick recap) • ER vs UML — understanding where		

	each fits		
5	**ER Diagram (Part 1)** · Entities, relationships, cardinality, and constraints · Mapping use cases to ER draft and validating with sample queries **ER Diagram (Part 2)** · Refinement — many-to-many relationships, surrogate keys, and constraints · Converting ER diagrams to tables; schema evolution and simple migrations		
6	**Patterns (Intro & Creational)** · Why design patterns — understanding forces and trade-offs · Singleton and Factory patterns — lifecycle, testability, and DI-friendly singletons **Patterns (Structural)** · Adapter pattern — wrapping and integrating third-party APIs · Composite pattern — representing and managing tree structures		

7	**Patterns (Behavioral I)** • Observer pattern — decoupled event handling and simple fan-out at code level **Patterns (Behavioral II)** • Strategy pattern — pluggable behaviors and comparison with Template Method pattern		
8	**SOLID (Part 1)** • Single Responsibility Principle (SRP) — organizing responsibilities clearly • Open/Closed Principle (OCP) — creating seams for extension **SOLID (Part 2)** • Liskov Substitution Principle (LSP) — ensuring valid substitution in hierarchies • Interface Segregation Principle (ISP) — designing focused and minimal interfaces • Dependency Inversion Principle (DIP) — depending on abstractions, not concretions • Aligning SOLID with design patterns — Factory + DIP, Observer + ISP		

9	**Machine Coding to MVP** · Vertical slice scoping — defining a minimal yet functional feature flow · Writing unit tests first — covering happy and sad paths · Skeleton implementation guided by tests — using stubs and fakes **LLM-Assisted Coding (Responsible Use)** · Translating requirements into effective prompts and constraining outputs · Verifying generated code with tests for correctness and safety · Notebook-style exploratory workflow — quick spikes leading to production-ready code		
10	**Code Organization and Modularization (Part 1)** · Importance of code organization · Modularization techniques — Separation of concerns and layered architecture · Microservices vs Monolithic architecture — trade-offs and considerations · Best practices for writing maintainable		

	and scalable code **Process and Collaboration** • SDLC overview — Waterfall, Agile, and DevOps basics • Software process models — Scrum, Kanban, and XP fundamentals • Project management and collaboration tools — Jira and Linear • Version control workflows and code reviews • Coding standards and best practices		
11	**Sample Test Cases for Machine Coding**		
12	**Sample Test Cases for Machine Coding**		

6.0 EVALUATION CRITERIA

Map the percentage weightage to each category.

Function	% Weightage	Notes
Attendance/Class Participation	**5%**	
In Class/Post Class Assignments	**10%**	
Mini-projects	**10%**	
Contests	**15%**	
Mid Semester	**20%**	
End Semester	**40 %**	

7.0 REQUIRED READINGS AND REFERENCES

List of required and suggested material. Can mention week-wise or topic-wise. Can also mention how to access readings.

Readings	1. *Head First Design Patterns* - Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra - O'Reilly Media 2. *Clean Architecture* – Robert C. Martin, Prentice Hall 3. *Software Engineering* – Ian Sommerville, Pearson Education
Videos	University level series: **Lecture - 1 Introduction to Software Engineering** by IIT Bombay - link
Others	1. *Design Patterns: Elements of Reusable Object-Oriented Software* – Erich Gamma et al., Addison-Wesley 2. *Clean Code: A Handbook of Agile Software Craftsmanship* – Robert C. Martin, Prentice Hall 3. *Continuous Delivery* – Jez Humble & David Farley, Addison-Wesley 4. *Site Reliability Engineering* – Google SRE Team, O'Reilly Media 5. *Designing Data-Intensive Applications* by Martin Kleppmann, O'Reilly Media

8.0 PROGRAM AUTHORIZATION

(SIGNOFF)

The undersigned acknowledge they have reviewed the course outline. Changes to this course outline will be coordinated with and approved by the undersigned or their designated representatives.

Program Head	Signature: _____ Date: _____ Print Name: _____ Title: _____

	Role:
	Signature: Date: Print Name: Title: Role:
OAA Office	Signature: Date: Print Name: Title: Role:
VC Office	Signature: Date: Print Name: Title: Role:
	Signature: Date: Print Name: Title: Role:
Course Owner	Signature: Date: Print Name: Title: Role:
	Signature: Date: Print Name:

	Title: _____ Role: _____
	Signature: _____ Date: _____ Print Name: _____ Title: _____ Role: _____